SKYBOOKER

Detailed Class Diagram Documentation

Airline Ticket Booking System · Microservices Architecture

Architecture	Services	Stack	Database
Microservices	9 Core Services	Spring Boot 3.2 / Java 17	PostgreSQL (per service)

TABLE OF CONTENTS

1. PROJECT OVERVIEW & ARCHITECTURE

SkyBooker is a comprehensive airline ticket booking platform built on a microservices architecture using **Spring Boot 3.2** and **Java 17**. The system decomposes its domain into nine independently deployable services, each owning its own database and exposing a well-defined REST API. Services communicate synchronously via HTTP (RestTemplate / WebClient) and asynchronously through **RabbitMQ** message queues.

Principle	Implementation
Database per Service	Each microservice owns a dedicated PostgreSQL database, ensuring loose coupling and independent scalability.
Single Responsibility	Every service handles one bounded context: auth, flights, seats, bookings, passengers, payments, notifications, airlines/airports, and admin.
REST-First Communication	Synchronous inter-service calls use RestTemplate/WebClient; async events flow through RabbitMQ.
Layered Architecture	Controller to Service to Repository to Database per service, enforcing separation of concerns.
Security	JWT-based authentication issued by Auth Service; roles: PASSENGER, ADMIN, AIRLINE_STAFF.
Observability	SonarQube for code quality, JaCoCo for coverage, and an AuditLog entity for administrative traceability.

2. MICROSERVICES SUMMARY

Auth Service
Port: 8081

Flight Service
Port: 8082

Seat Service
Port: 8083

Booking Service
Port: 8084

Passenger Service
Port: 8085

Payment Service
Port: 8086

Notification Service
Port: 8087

Airline-Airport Service
Port: 8088

Admin-User Service
Port: 8089

Service	Port	Database	Primary Entity	Responsibility
Auth Service	8081	auth_db	User	User registration, JWT auth, OAuth 2.0 (Google)
Flight Service	8082	flight_db	Flight	Flight CRUD, availability, status management
Seat Service	8083	seat_db	Seat	Seat inventory, hold/release, seat assignment
Booking Service	8084	booking_db	Booking	Booking lifecycle, PNR generation, check-in
Passenger Service	8085	passenger_db	Passenger	Passenger details per booking, travel documents
Payment Service	8086	payment_db	Payment	Payment processing, Razorpay integration, refunds
Notification Service	8087	notification_db	Notification	Email/SMS/push notifications via RabbitMQ
Airline-Airport Service	8088	airline_airport_db	Airline/Airport	Airline & airport registry, IATA codes
Admin-User Service	8089	admin_db	AdminReport	Reports, analytics, audit logging, user management

3. AUTH SERVICE – USER ENTITY

The Auth Service is the identity backbone of SkyBooker. It manages user registration, login, password reset, and Google OAuth 2.0 integration. Every API request carries a JWT token issued by this service. Role-based access control distinguishes PASSENGER, ADMIN, and AIRLINE_STAFF users.

Fields

Field	Type	Constraints
id	`Long`	PK · Auto-generated
firstName	`String`	NOT NULL · 2–50 chars
lastName	`String`	NOT NULL · 2–50 chars
email	`String`	NOT NULL · UNIQUE · max 100 chars
password	`String`	Nullable (OAuth users have no password)
phoneNumber	`String`	Nullable · exactly 10 digits
passportNumber	`String`	Nullable · max 20 chars
nationality	`String`	Nullable · max 50 chars
profilePhotoUrl	`String`	Nullable · max 1000 chars
airlineId	`Long`	FK → Airline (nullable; set for AIRLINE_STAFF)
googleId	`String`	UNIQUE · Nullable · OAuth identifier
authProvider	`AuthProvider`	ENUM · LOCAL or GOOGLE
role	`UserRole`	ENUM · PASSENGER / ADMIN / AIRLINE_STAFF
isActive	`Boolean`	Default: true (soft-delete flag)
resetToken	`String`	Nullable · max 100 chars · password reset
resetTokenExpiry	`LocalDateTime`	Nullable · expiry for reset token
createdAt	`LocalDateTime`	NOT NULL · auto-set on @PrePersist
updatedAt	`LocalDateTime`	NOT NULL · auto-updated on @PreUpdate

Enumerations

Enum Name	Values
UserRole	`PASSENGER, ADMIN, AIRLINE_STAFF`
AuthProvider	`LOCAL, GOOGLE`

Relationships

Relationship	Target	Description
ManyToOne	Airline	User belongs to one Airline (airline staff only)
OneToMany	Booking	A user can have many bookings
OneToMany	Payment	A user can have many payment records
OneToMany	Notification	A user receives many notifications

4. FLIGHT SERVICE – FLIGHT ENTITY

The Flight Service manages the flight schedule. Each flight references an Airline and two Airport entities (departure and arrival) by ID. It tracks seat availability and fare and exposes status transitions (ON_TIME → DELAYED → CANCELLED etc.). The Booking and Seat services query this service to validate availability before committing a reservation.

Fields

Field	Type	Constraints
id	`Long`	PK · Auto-generated
flightNumber	`String`	NOT NULL · UNIQUE · max 10 chars
aircraftType	`String`	NOT NULL · max 10 chars
airlineId	`Long`	NOT NULL · FK → Airline
departureAirportId	`Long`	NOT NULL · FK → Airport
arrivalAirportId	`Long`	NOT NULL · FK → Airport
departureTime	`LocalDateTime`	NOT NULL
arrivalTime	`LocalDateTime`	NOT NULL
totalSeats	`Integer`	NOT NULL · total seat capacity
availableSeats	`Integer`	NOT NULL · decrements on booking
baseFare	`BigDecimal`	NOT NULL · precision=10, scale=2
status	`FlightStatus`	NOT NULL · ENUM
createdAt	`LocalDateTime`	NOT NULL · auto-set
updatedAt	`LocalDateTime`	NOT NULL · auto-updated

Enumerations

Enum Name	Values
FlightStatus	`ON_TIME, DELAYED, CANCELLED, DEPARTED, ARRIVED`

Relationships

Relationship	Target	Description
ManyToOne	Airline	Flight is operated by one Airline
ManyToOne	Airport	departureAirportId → departure airport
ManyToOne	Airport	arrivalAirportId → arrival airport
OneToMany	Seat	A flight has many seats
OneToMany	Booking	A flight can have many bookings

5. SEAT SERVICE – SEAT ENTITY

The Seat Service manages per-flight seat inventory with a hold/release mechanism. When a user initiates booking, seats are temporarily HELD (with a holdExpiresAt TTL). On payment success the seat transitions to BOOKED. Expired holds are reclaimed by a scheduled job, ensuring no seat is permanently locked by an abandoned transaction.

Fields

Field	Type	Constraints
id	`Long`	PK · Auto-generated
flightId	`Long`	NOT NULL · FK → Flight
seatNumber	`String`	NOT NULL · max 10 chars (e.g. 12A)
seatClass	`SeatClass`	NOT NULL · ENUM
status	`SeatStatus`	NOT NULL · ENUM
passengerId	`Long`	Nullable · FK → Passenger (set on check-in)
bookingId	`Long`	Nullable · FK → Booking
holdExpiresAt	`LocalDateTime`	Nullable · when HELD state expires
createdAt	`LocalDateTime`	NOT NULL · auto-set
updatedAt	`LocalDateTime`	NOT NULL · auto-updated

Enumerations

Enum Name	Values
SeatClass	`ECONOMY, BUSINESS, FIRST`
SeatStatus	`AVAILABLE, HELD, BOOKED, UNAVAILABLE`

Relationships

Relationship	Target	Description
ManyToOne	Flight	Seat belongs to one Flight (UNIQUE constraint: flightId+seatNumber)
ManyToOne	Passenger	Seat assigned to one Passenger after check-in
ManyToOne	Booking	Seat is associated with one Booking

6. BOOKING SERVICE – BOOKING ENTITY

The Booking Service is the central orchestrator of the booking workflow. It generates a unique PNR (Passenger Name Record) and coordinates with the Flight, Seat, Payment, and Notification services. Fare is broken down into baseFare, taxes, and ancillaryCharges. Selected seat numbers are stored as a JSON array in selectedSeatsJson, with helper methods for serialisation.

Fields

Field	Type	Constraints
id	`Long`	PK · Auto-generated
pnr	`String`	NOT NULL · UNIQUE · Indexed · max 20 chars
userId	`Long`	NOT NULL · FK → User
flightId	`Long`	NOT NULL · FK → Flight
numberOfPassengers	`Integer`	NOT NULL
baseFare	`BigDecimal`	NOT NULL · precision=10, scale=2
taxes	`BigDecimal`	NOT NULL · precision=10, scale=2
ancillaryCharges	`BigDecimal`	NOT NULL · precision=10, scale=2
totalFare	`BigDecimal`	NOT NULL · sum of fare components
totalAmount	`BigDecimal`	NOT NULL · final charged amount
status	`BookingStatus`	NOT NULL · ENUM
bookingDate	`LocalDateTime`	NOT NULL
paymentId	`Long`	Nullable · FK → Payment
checkedIn	`Boolean`	NOT NULL · default: false
checkedInAt	`LocalDateTime`	Nullable
selectedSeatsJson	`String (TEXT)`	JSON array of seat numbers
checkInReminderSent	`Boolean`	NOT NULL · default: false
createdAt	`LocalDateTime`	NOT NULL · auto-set
updatedAt	`LocalDateTime`	NOT NULL · auto-updated

Enumerations

Enum Name	Values
BookingStatus	`PENDING, CONFIRMED, CANCELLED, COMPLETED, NO_SHOW`

Relationships

Relationship	Target	Description
ManyToOne	User	Booking belongs to one User
ManyToOne	Flight	Booking is for one Flight
OneToOne	Payment	Booking linked to one Payment record
OneToMany	Passenger	Booking contains one or more Passengers

OneToMany	Notification	Booking triggers many Notifications

7. PASSENGER SERVICE – PASSENGER ENTITY

The Passenger Service records personal and travel-document details for each traveller within a booking. A single booking can span multiple passengers (adults, children, infants). Passport and nationality fields support international travel compliance.

Fields

Field	Type	Constraints
id	`Long`	PK · Auto-generated
bookingId	`Long`	NOT NULL · FK → Booking
firstName	`String`	NOT NULL · max 100 chars
lastName	`String`	NOT NULL · max 100 chars
email	`String`	Nullable · max 255 chars
phoneNumber	`String`	Nullable · max 20 chars
passportNumber	`String`	NOT NULL · max 20 chars
dateOfBirth	`LocalDate`	NOT NULL
category	`Category`	NOT NULL · ENUM
gender	`Gender`	NOT NULL · ENUM
nationality	`String`	NOT NULL · max 50 chars
specialRequests	`String`	Nullable · max 50 chars
createdAt	`LocalDateTime`	NOT NULL · auto-set
updatedAt	`LocalDateTime`	NOT NULL · auto-updated

Enumerations

Enum Name	Values
Gender	`MALE, FEMALE, OTHER`
Category	`ADULT, CHILD, INFANT`

Relationships

Relationship	Target	Description
ManyToOne	Booking	Passenger belongs to one Booking
OneToMany	Seat	A passenger is assigned seats (via Seat.passengerId)

8. PAYMENT SERVICE – PAYMENT ENTITY

The Payment Service integrates with Razorpay to handle online payments. Each transaction is tracked with a unique transactionId and can carry both a gatewayTransactionId and Razorpay-specific order/payment IDs. Refunds update refundAmount and set status to REFUNDED.

Fields

Field	Type	Constraints
id	`Long`	PK · Auto-generated
transactionId	`String`	NOT NULL · UNIQUE · max 50 chars
bookingId	`Long`	NOT NULL · FK → Booking
userId	`Long`	NOT NULL · FK → User
amount	`BigDecimal`	NOT NULL · precision=10, scale=2
paymentMethod	`PaymentMethod`	NOT NULL · ENUM
status	`PaymentStatus`	NOT NULL · ENUM
gatewayTransactionId	`String`	Nullable · max 100 chars
razorpayOrderId	`String`	Nullable · max 100 chars
razorpayPaymentId	`String`	Nullable · max 100 chars
failureReason	`String`	Nullable · max 500 chars
transactionDate	`LocalDateTime`	NOT NULL
refundAmount	`BigDecimal`	Nullable · precision=10, scale=2
createdAt	`LocalDateTime`	NOT NULL · auto-set
updatedAt	`LocalDateTime`	NOT NULL · auto-updated

Enumerations

Enum Name	Values
PaymentMethod	`CREDIT_CARD, DEBIT_CARD, NET_BANKING, WALLET, UPI`
PaymentStatus	`PENDING, SUCCESS, FAILED, REFUNDED, CANCELLED`

Relationships

Relationship	Target	Description
ManyToOne	Booking	Payment belongs to one Booking
ManyToOne	User	Payment initiated by one User

9. NOTIFICATION SERVICE – NOTIFICATION ENTITY

The Notification Service consumes events from RabbitMQ and dispatches messages to users via multiple channels. Each notification tracks delivery status (PENDING → SENT → DELIVERED or FAILED). The isRead flag supports in-app notification centre functionality.

Fields

Field	Type	Constraints
id	`Long`	PK · Auto-generated
userId	`Long`	NOT NULL · FK → User
bookingId	`Long`	NOT NULL · FK → Booking
type	`NotificationType`	NOT NULL · ENUM · max 50 chars
channel	`Channel`	NOT NULL · ENUM · max 20 chars
subject	`String`	NOT NULL · max 500 chars
message	`String`	NOT NULL · max 2000 chars
status	`NotificationStatus`	NOT NULL · ENUM
isRead	`Boolean`	NOT NULL · default: false
recipient	`String`	Nullable · email or phone number
createdAt	`LocalDateTime`	NOT NULL · auto-set
updatedAt	`LocalDateTime`	NOT NULL · auto-updated

Enumerations

Enum Name	Values
NotificationType	`BOOKING_CONFIRMATION, PAYMENT_SUCCESS, CHECK_IN_REMINDER, FLIGHT_UPDATE, CANCELLATION, REFUND`
Channel	`EMAIL, SMS, IN_APP, PUSH_NOTIFICATION`
NotificationStatus	`PENDING, SENT, DELIVERED, FAILED, UNDELIVERABLE`

Relationships

Relationship	Target	Description
ManyToOne	User	Notification is sent to one User
ManyToOne	Booking	Notification is triggered by one Booking

10. AIRLINE-AIRPORT SERVICE

The Airline-Airport Service acts as the master data registry for airlines and airports. Both entities expose an IATA code and contact details. The isActive flag enables soft-delete. All other services reference airlines and airports by foreign key only.

10a. Airline Entity

Field	Type	Constraints
id	`Long`	PK · Auto-generated
name	`String`	NOT NULL · UNIQUE · max 100 chars
iataCode	`String`	NOT NULL · UNIQUE · max 10 chars
description	`String`	Nullable · max 2000 chars
phoneNumber	`String`	Nullable · max 20 chars
email	`String`	Nullable · max 100 chars
isActive	`Boolean`	NOT NULL · default: true
createdAt	`LocalDateTime`	NOT NULL · auto-set
updatedAt	`LocalDateTime`	NOT NULL · auto-updated

Relationship	Target	Description
OneToMany	Flight	An airline operates many flights (via Flight.airlineId)
OneToMany	User	An airline has many staff users (via User.airlineId)

10b. Airport Entity

Field	Type	Constraints
id	`Long`	PK · Auto-generated
name	`String`	NOT NULL · UNIQUE · max 100 chars
iataCode	`String`	NOT NULL · UNIQUE · max 10 chars
city	`String`	NOT NULL · max 50 chars
country	`String`	NOT NULL · max 50 chars
description	`String`	Nullable · max 2000 chars
phoneNumber	`String`	Nullable · max 20 chars
email	`String`	Nullable · max 100 chars
isActive	`Boolean`	NOT NULL · default: true
createdAt	`LocalDateTime`	NOT NULL · auto-set
updatedAt	`LocalDateTime`	NOT NULL · auto-updated

Relationship	Target	Description
OneToMany	Flight (departure)	Airport is departure point for many flights
OneToMany	Flight (arrival)	Airport is arrival point for many flights

11. ADMIN-USER SERVICE

The Admin-User Service provides operational dashboards and audit trails. AdminReport entities are generated periodically (daily, weekly, monthly, yearly, or custom) and aggregate revenue, booking, flight, and user metrics across the entire platform. AuditLog records every administrative action with actor, target, and detail fields.

11a. AdminReport Entity

Field	Type	Constraints
id	`Long`	PK · Auto-generated
reportName	`String`	NOT NULL · max 100 chars
description	`String`	Nullable · max 500 chars
reportType	`ReportType`	NOT NULL · ENUM
totalRevenue	`BigDecimal`	NOT NULL · precision=15, scale=2
totalBookings	`Long`	NOT NULL · count of bookings in period
totalFlights	`Long`	NOT NULL · count of flights in period
activeUsers	`Long`	NOT NULL · active user count
createdAt	`LocalDateTime`	NOT NULL · auto-set
updatedAt	`LocalDateTime`	NOT NULL · auto-updated

Enum Name	Values
ReportType	`DAILY, WEEKLY, MONTHLY, YEARLY, CUSTOM`

11b. AuditLog Entity

Field	Type	Constraints
id	`Long`	PK · Auto-generated
actor	`String`	NOT NULL · max 100 chars · who performed the action
action	`String`	NOT NULL · max 100 chars · what was done
targetType	`String`	NOT NULL · max 100 chars · entity type affected
targetId	`String`	Nullable · max 100 chars · entity ID affected
details	`String`	Nullable · max 1000 chars · extra context
createdAt	`LocalDateTime`	NOT NULL · auto-set (no updatedAt – immutable)

12. ENTITY RELATIONSHIP SUMMARY

Entity	Relationship	Related Entity	Via Field	Cardinality
User	ManyToOne	Airline	User.airlineId	Many users → 1 airline
User	OneToMany	Booking	Booking.userId	1 user → many bookings
User	OneToMany	Payment	Payment.userId	1 user → many payments
User	OneToMany	Notification	Notification.userId	1 user → many notifications
Airline	OneToMany	Flight	Flight.airlineId	1 airline → many flights
Airline	OneToMany	User	User.airlineId	1 airline → many staff
Airport	OneToMany	Flight (dep)	Flight.departureAirportId	1 airport → many departures
Airport	OneToMany	Flight (arr)	Flight.arrivalAirportId	1 airport → many arrivals
Flight	OneToMany	Seat	Seat.flightId	1 flight → many seats
Flight	OneToMany	Booking	Booking.flightId	1 flight → many bookings
Booking	ManyToOne	User	Booking.userId	Many bookings → 1 user
Booking	ManyToOne	Flight	Booking.flightId	Many bookings → 1 flight
Booking	OneToOne	Payment	Booking.paymentId	1 booking → 1 payment
Booking	OneToMany	Passenger	Passenger.bookingId	1 booking → many passengers
Booking	OneToMany	Notification	Notification.bookingId	1 booking → many notifications
Seat	ManyToOne	Flight	Seat.flightId	Many seats → 1 flight
Seat	ManyToOne	Passenger	Seat.passengerId	Many seats → 1 passenger
Seat	ManyToOne	Booking	Seat.bookingId	Many seats → 1 booking
Passenger	ManyToOne	Booking	Passenger.bookingId	Many passengers → 1 booking
Payment	ManyToOne	Booking	Payment.bookingId	Many payments → 1 booking
Notification	ManyToOne	User	Notification.userId	Many notifications → 1 user
Notification	ManyToOne	Booking	Notification.bookingId	Many notifications → 1 booking

13. DATABASE SCHEMA OVERVIEW

SkyBooker follows the **Database per Service** pattern. Each microservice has its own isolated PostgreSQL database. Cross-service data references are resolved through foreign-key IDs propagated over REST calls — there are no shared database connections between services.

Database	Tables	Notes
`auth_db`	users	Stores credentials, roles, OAuth data, reset tokens
`airline_airport_db`	airlines, airports	Master data; seeded at startup
`flight_db`	flights	Mutable — status updated by airline staff
`seat_db`	seats	Hold expiry enforced by scheduler
`booking_db`	bookings (index on pnr)	PNR is the external booking reference
`passenger_db`	passengers	One row per traveller per booking
`payment_db`	payments	Razorpay IDs stored for reconciliation
`notification_db`	notifications	Consumed from RabbitMQ; delivery status tracked
`admin_db`	admin_reports, admin_audit_logs	Reports generated by scheduled jobs

14. SERVICE LAYER ARCHITECTURE

Every microservice is structured into four horizontal layers. This enforces separation of concerns and makes each layer independently testable.

Controller Layer	REST endpoints, request/response handling, input validation (@Valid), HTTP status mapping.
Service Layer	Business logic, transaction management (@Transactional), inter-service REST calls (WebClient / RestTemplate).
Repository Layer	Spring Data JPA repositories, custom JPQL queries, pagination and sorting.
Database Layer	PostgreSQL database. Each service has its own schema, accessed only by its own Repository layer.

15. INTER-SERVICE COMMUNICATION FLOW

Services exchange data in two ways: **synchronous REST calls** for immediate responses and **asynchronous RabbitMQ events** for fire-and-forget operations such as notifications. The booking creation flow illustrates both patterns:

Step	From → To	Description
1	**User → Booking Service**	POST /bookings — user submits booking request with flightId, seat selection, and passenger details.
2	**Booking → Flight Service**	GET /flights/{id} — verify flight exists, status is not CANCELLED, and availableSeats ≥ requested count.
3	**Booking → Seat Service**	POST /seats/hold — temporarily mark selected seats as HELD with a TTL (holdExpiresAt).
4	**Booking → Payment Service**	POST /payments — create a Razorpay order; return orderId to the client for frontend payment capture.
5	**Payment → Booking Service**	PATCH /bookings/{id}/confirm — on Razorpay webhook success, confirm booking status → CONFIRMED.
6	**Booking → Seat Service**	POST /seats/book — transition held seats to BOOKED state.
7	**Booking → Flight Service**	PATCH /flights/{id}/seats — decrement availableSeats count.
8	**Payment → RabbitMQ**	Publish PAYMENT_SUCCESS event to notification.exchange queue.
9	**RabbitMQ → Notification Service**	Consume event; send booking confirmation email + SMS to passenger.

16. SECURITY & DESIGN PATTERNS

Pattern / Mechanism	Details
JWT Authentication	Auth Service issues signed JWTs. All other services validate the token on every request via a shared secret or public key.
Role-Based Access Control	Three roles: PASSENGER (booking/viewing), ADMIN (full access), AIRLINE_STAFF (manage own airline flights and seats).
OAuth 2.0 - Google	Users can register and login with Google. googleId stored on User; password nullable for OAuth users.
BCrypt Password Hashing	Passwords hashed with BCrypt before persistence; raw passwords are never stored.
CORS Configuration	Allowed origins configured per environment; preflight OPTIONS requests handled globally.
Global Exception Handling	@ControllerAdvice maps domain exceptions to standardised RFC-7807 problem details responses.
Soft Delete	isActive flag on User, Airline, and Airport allows logical deletion without data loss.
Audit Logging	Every admin action writes an AuditLog row (actor, action, targetType, targetId, details).
Seat Hold TTL	Seat.holdExpiresAt prevents indefinite holds; a @Scheduled job reclaims expired holds.
DTO Pattern	Request DTOs validated with Jakarta Validation (@NotNull, @Size, @Email). Response DTOs hide internal entity structure.
Repository Pattern	Spring Data JPA; custom JPQL methods for complex queries (e.g. findByPnr, findByFlightIdAndStatus).
Transactional Boundaries	@Transactional on service methods; distributed transactions avoided by eventual-consistency via events.

17. TECHNOLOGY STACK

Category	Technology	Version / Notes
Language	Java	17 (LTS)
Framework	Spring Boot	3.2.0
Cloud / Microservices	Spring Cloud	2023.0.0
Persistence	Spring Data JPA / Hibernate	Included in Spring Boot
Security	Spring Security	JWT + OAuth 2.0
Database	PostgreSQL	One instance per service
Messaging	RabbitMQ	Async event bus for notifications
Payment Gateway	Razorpay	Order creation + webhook verification
JSON Processing	Jackson	ObjectMapper for selectedSeatsJson
Boilerplate	Lombok	@Data, @Builder, @Slf4j, etc.
Build Tool	Maven	Multi-module POM
Testing	JUnit 5 + Mockito	Unit & integration tests
Test Support	Spring Boot Test	@SpringBootTest, MockMvc
Code Quality	SonarQube	Static analysis, smell detection
Coverage	JaCoCo	Coverage reports in CI